

Atividade 2: Auditoria Forense e Plano de Resgate Técnico

Projeto: langfuse/langfuse

Equipe: Dean Vinícius Palmeira de Meneses; Hilton Hunaldo Costa Silva Brito

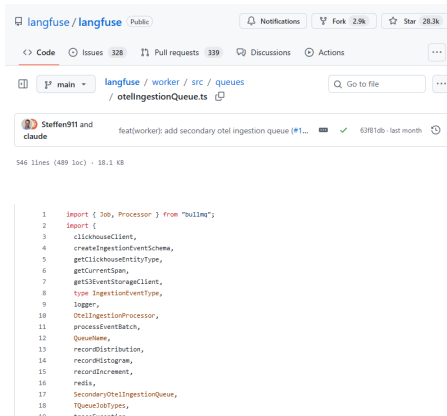
2026-05-31

Principais riscos identificados

- Acoplamento de providers de IA em multiplas camadas (alto impacto para troca).
- TODOs em filas criticas (ingestao e webhooks) indicando riscos latentes.
- Orquestracao centralizada em um unico fluxo (SRP fragil).

Eixo A: Gestao (MPS.BR GPR)

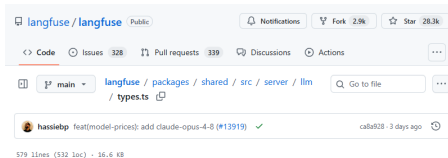
- Evidencia de investigacao tecnica e mudanca de escopo em issue #13933.
- Riscos ocultos em TODOs de filas e LLM.
- Cadencia de commits consistente, mas revisao humana nem sempre explicita.



```
1 import { Job, Processor } from "bullmq";
2 import {
3   clickhouseClient,
4   createIngestionEventSchema,
5   getClickhouseEntityType,
6   getCurrentSpan,
7   getS3EventStorageClient,
8   type IngestionEventType,
9   logger,
10  OtelIngestionProcessor,
11  processEventBatch,
12  QueueName,
13  recordDistribution,
14  recordHistogram,
15  recordIncrement,
16  redis,
17  SecondaryOtelIngestionQueue,
18  TQueueJobTypes,
19  type IngestionEvent
```

Eixo B: Anatomia do Código (SOLID/DRY)

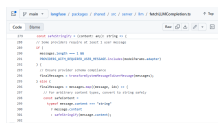
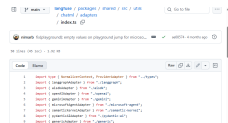
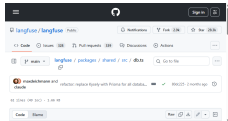
- Teste da troca mostra logica espalhada por shared/server/UI.
- “God object”: fetchLLMCompletion concentra responsabilidades.
- DRY violado em duplicacao de tree building.



```
1 import { LlmApiKeys } from "@prisma/client";
2 import z from "zod";
3 import {
4   BedrockConfigSchema,
5   VertexAIConfigSchema,
6 } from "../../interfaces/customLLMProviderConfigSchemas";
7 import { JSONObjectSchema } from "../../utils/zod";
8 import type {
9   InternalTraceEventInput,
10   InternalTraceExperimentContext,
11 } from "../../internalTraceEvents";
12
13 // disable lint as this is exported and used in web/worker
14
15 export const LLMJSONSchema = z.record(z.string(), z.any());
16 export type LLMJSONSchema = z.infer<typeof LLMJSONSchema>;
17
18 export const JSONSchemaFormSchema = z
19   .string()
20   .transform((value, ctx) => {
21     try {
22       const parsed = JSON.parse(value);
23       return parsed;
24     } catch {
25       ctx.addIssue({
26         code: "custom"
```

Eixo C: Padroes GoF

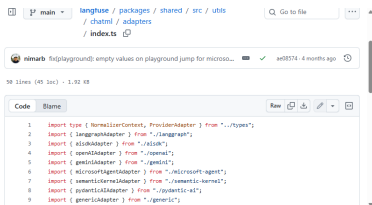
- Criacionais: Singleton (PrismaClient, SlackService).
- Estruturais: Adapter para normalizacao de frameworks.
- Comportamentais: ausencia de Strategy/Chain centralizado.



- Refatoracao conceitual: Strategy + Registry por provider.
- Roadmap MPS.BR G:
 - Planejamento com baseline e milestones.
 - Rastreabilidade requisito -> issue -> PR -> teste.
 - Registro de riscos + politica de code review.

Refatoracao conceitual (exemplo)

```
interface LLMProviderStrategy { ... }  
registry.get(adapter).buildClient(...)
```



The screenshot shows a code editor interface. At the top, a file explorer shows a tree structure with folders like 'langfuse', 'packages', 'shared', 'src', 'utils', 'chatml', 'adapters', and 'index.ts'. Below the explorer, a search bar contains 'Go to file'. A notification bar shows a user profile for 'nimarb' and a message about playground values. The main code area displays a TypeScript file with imports and interface definitions. The code is as follows:

```
1 import type { NormalizerContext, ProviderAdapter } from "../types";  
2 import { LanggraphAdapter } from "../langgraph";  
3 import { AISDKAdapter } from "../aiSdk";  
4 import { OpenAIAdapter } from "../openai";  
5 import { GeminiAdapter } from "../gemini";  
6 import { MicrosoftAgentAdapter } from "../microsoft-agent";  
7 import { SemanticKernelAdapter } from "../semantic-kernel";  
8 import { PydanticAIAdapter } from "../pydantic-ai";  
9 import { GenericAdapter } from "../generic";  
--
```

- Resultado: riscos priorizados e plano de resgate alinhado ao MPS.BR.
- Proximo passo: executar as 3 acoes iniciais e medir reducao de risco.